

Chess Engine

Nikola Pantoić RA135/2023

3.6.2026.

1 Definicija problema

Problem počinje kao vrlo jednostavna ideja, pronaći najbolji mogući potez u šahovskoj partiji. Iako se na prvi pogled čini kao jednostavan problem, zapravo je veoma složen i zahteva duboko razumevanje igre šaha kao i efikasnih algoritma za pretragu i procenu pozicija. Nakon samo šest poteza, broj mogućih pozicija u šahu prelazi 120.9 miliona. Do sada niko nije uspeo da nadje način za pronalaženje najboljeg poteza u svim mogućim pozicijama. Cilj ovog projekta je da se uporedi sa postojećim šahovskim engine-ima i da istraži različite algoritme pretrage i evaluacije pozicija, uključujući i upotrebu neuronskih mreža (NNUE).

2 Skup Podataka

Za treniranje neuronske mreže biće korišćen skup podataka dostupan na Kaggle web sajtu: <https://www.kaggle.com/datasets/mateuszgrzybpl/lichess-chess-positions-ml-ready-and-deduplicated>. Ovaj dataset predstavlja smanjenu varijantu ogromnog lichess dataseta. Ima 316,072,343 jedinstvenih pozicija od kojih će samo deo biti korišćen. Sadrži četiri kolone:

1. **fen** (Forsyth-Edwards Notation) - Reprezentacija trenutnog stanja table u FEN formatu. Predstavlja **ulaz** u neuronsku mrežu. FEN string se konvertuje u **HalfKP** format. Za oba igrača (belog i crnog) generiše se zaseban retki (sparse) binarni vektor dimenzionalnosti 45,056 (64 polja za kralja $\times$ 11 tipova ostalih figura $\times$ 64 polja).
2. **depth** - Stockfish dubina pretrage
3. **cp** - Centipawn evaluacija pozicije od strane Stockfish engine-a (null ako je pozicija mat). Predstavlja **ciljnu promeljivu** za treniranje neuronske mreže. Za pozicije ne možemo garantovati da su najbolje ocenjene, Stockfish (kao trenutno najbolji šahovski engine) evaluacija se koristi kao aproksimacija "istine" u procesu treniranja. Više o centipawnima u odeljku Evaluacija.
4. **mate** - Broj poteza do mat pozicije (ako postoji)

Format dataseta je `.parquet` i podeljen je u 10 datoteka

3 Metodologija

Svaka partija počinje u stanju početne pozicije u šahu.

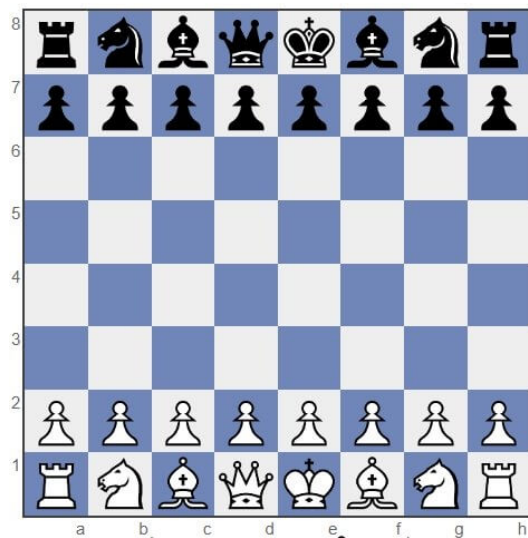


Figure 1: Početna pozicija

Beli igrač započinje partiju nakon čega igrači igraju naizmenično sve do terminalnog stanja (kralj ne može da se odbrani ili neki od igrača nema legalnih poteza). Stanje table i legalni potezi biće predstavljeni pomoću `python-chess` biblioteke. Ulogu igrača će imati ageni koji će koristiti različite algoritme za pronalaženje i evaluaciju najboljeg poteza. Postojeće pet različitih agenata koji će se međusobno takmičiti i rezultati će biti upoređeni:

- **Agent 1:** Koristi PVS (Principal Variation Search) koji je optimizacija Alpha-Beta Pruning (Minimax) algoritma za pretragu. Evaluacija pozicija će se vršiti pomoću jednostavne funkcije evaluacije koja uzima u obzir materijalnu prednost, kontrolu centra, sigurnost kralja i mobilnost figura.
- **Agent 2:** Koristi MCTS (Monte Carlo Tree Search) koji je algoritam zasnovan na simulacijama. U sklopu njega će se koristiti UCB1 (Upper Confidence Bounds) heuristika.
- **Agent 3:** PVS + PyTorch NNUE (Efficiently Updatable Neural Network) napravljen uz pomoć PyTorch biblioteke i treniran na našem datasetu. Mreža koristi klasičnu **HalfKP** arhitekturu sa deljenim težinama (Shared Weights). Struktura mreže:

- **Ulazni sloj:** Retki (sparse) binarni vektor dimenzionalnosti 45,056. Pozicija se posmatra iz dve perspektive: *Side to move* (igrač na potezu) i *Side not to move* (pasivan igrač). Oba ulaza dele isti prostor težina. Originalni HalfKP ima 41,024 inputa, ali cemo koristiti prošireni set od 45,056 koji uključuje dodatne poteze.
 - **Skriveni sloj 1 (Accumulator):** 512 (2x256) neurona. Jedan par od 256 neurona odgovara svakom ulazu. Squared Clipped ReLU aktivacija
 - **Skriveni sloj 2:** 32 neurona. Squared Clipped ReLU aktivacija
 - **Skriveni sloj 3:** 32 neurona. Squared Clipped ReLU aktivacija
 - **Izlazni sloj:** 1 neuron sa linearnom aktivacijom koji direktno vraća vrednost evaluacije pozicije.
- **Agent 4:** PVS + dosta napredniji Stockfish NNUE koji će takodje biti treniran na našem datasetu. Cilj ovog agenta je da se uporedi performanse i efikasnost našeg PyTorch NNUE modela sa već postojećim Stockfish NNUE modelom.
 - **Agent 5:** PVS + Stockfish NNUE koji je već istreniran i optimizovan od strane zajednice (<https://github.com/official-stockfish/networks>). Od ovog agenta se očekuje da pruži najbolju performansu.

Algoritmi pretrage:

- **PVS (Principal Variation Search)** - Optimizacija Alpha-Beta Pruning algoritma koja se fokusira na pretragu najvažnijih poteza. Koristi tehniku "Null Window" koja sprovodi brzi true/false test na uskom intervalu i omogućava brže odbacivanje loših poteza.

```
// Pseudo Kod za PVS algoritam
enum NodeType {
    NonPV,
    PV
};

template<NodeType nodeType>
int search( int alpha, int beta, int depth ) {
    if( depth == 0 ) return quiesce(alpha, beta);
    constexpr bool PVNode = nodeType != NonPV;
    int bestValue = -inf;
    for ( all moves ) {
        make
        if ( first move ) {
            score = -search<nodeType>(-beta, -alpha, depth - 1);
        } else {
            score = -search<NonPV>(-alpha - 1, -alpha, depth - 1);
        }
    }
}
```

```

        if ( score > alpha && PVNode )
            score = -search<PV>(-beta, -alpha, depth - 1); // re-search
    }
    unmake
    if( score >= beta )
        return score;    // fail-soft beta-cutoff
    if( score > alpha ) {
        alpha = score; // alpha acts like max in MiniMax
    }
}

return bestValue;
}

```

- **MCTS (Monte Carlo Tree Search)** - Algoritam zasnovan na simulacijama koji koristi UCB1 heuristiku za balansiranje između istraživanja i eksploatacije.

$$UCB1(n) = \frac{U(n)}{N(n)} + c \sqrt{\frac{\ln PARENT(n)}{N(n)}}$$

gde je $N(n)$ broj simulacija za čvor n , $U(n)$ ukupna vrednost čvora n , c je konstanta (uobičajno postavljena na $\sqrt{2}$).

4 Evaluacija

Napomena o centipawnima:

Osnovna jedinica mere za chess engine-e je centipawn (cp):

$$\begin{aligned}
 1cp &= \frac{1}{100} \text{pešak} \\
 \text{pešak} &= 100cp \\
 \text{konj} &= 300 \sim 320cp \\
 \text{lovac} &= 330 \sim 340cp \\
 \text{top} &= 500cp \\
 \text{dama} &= 900cp
 \end{aligned}$$

Pozitivna vrednost centipawna ukazuje na prednost belih, dok negativna vrednost ukazuje na prednost crnih. Ova skala važi za sve agente koji koriste PVS (A1, A3, A4, A5), dok MCTS agent (A2) ne koristi centipawne već win probability iz rollout simulacija.

Evaluacione funkcije po agentu:

- **Agent 1 (PVS + ručna heuristika):** Evaluacija pozicija će se vršiti pomoću jednostavne funkcije evaluacije koja uzima u obzir materijalnu

prednost, kontrolu centra, sigurnost kralja i mobilnost figura. Svakom od ovih faktora će biti dodeljena određena težina, a ukupna evaluacija pozicije će biti zbir faktora izražen u centipawnima.

- **Agent 2 (MCTS):** Evaluacija pozicije ne koristi eksplicitnu evaluacionu funkciju niti centipawne. Umesto toga svaki čvor stabla čuva win probability — odnos pobeda i ukupnog broja poseta tog čvora. Ova vrednost se ažurira propagacijom rezultata nasumičnih rollout simulacija: za svaku simulaciju ishod se kodira kao 1.0 (pobeda belog), 0.0 (pobeda crnog) ili 0.5 (nereseno). Selekcija poteza vrši se UCB1 formulom koja balansira istraživanje i eksploataciju na osnovu ovih win probability vrednosti.
- **Agent 3 (PVS + PyTorch NNUE):** Evaluacija se vrši neuronskom mrežom strukture $\text{HalfKP}(45, 056) \rightarrow 2 \times 256 \rightarrow 32 \rightarrow 32 \rightarrow 1$. Mreža prima FEN poziciju koja se transformiše u HalfKP indekse. Iz deljene matrice težina (Feature Transformer) paralelno se mapiraju vektori za igrača na potezu (*Side to move*) i njegovog protivnika (*Side not to move*). Ovi vektori (svaki veličine 256) se spajaju u akumulator od 512 neurona. Nakon primene *Squared Clipped ReLU* aktivacije, podaci prolaze kroz preostale guste slojeve do izlaznog neurona koji daje procenu u centipawnima. Kao loss funkcija koristi se MSE (Mean Squared Error):

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

gde je y_i **ciljna** Stockfish evaluacija iz dataseta (kolona **cp**), a $\hat{y}_i$ predviđena vrednost mreže, obe izražene u centipawnima. Tokom treniranja prate se MSE i MAE (Mean Absolute Error) po epohama.

- **Agent 4 (PVS + dataset-trained Stockfish NNUE):** Evaluacija se vrši Stockfish NNUE mrežom treniranom uz pomoć `nnue-pytorch` biblioteke. Trening koristi parametar λ koji kontroliše odnos izmedju evaluacionog skora i ishoda partije kao trening cilja: $\lambda = 1.0$ znači oslanjanje isključivo na Stockfish evaluaciju pozicije, $\lambda = 0.0$ isključivo na ishod partije, a vrednosti izmedju pružaju proporcionalni miks oba. Mreža se koristi kao evaluator u PVS pretrazi — prima FEN poziciju i vraća evaluaciju u centipawnima.
- **Agent 5 (PVS + pre-trained Stockfish NNUE):** Ista infrastruktura kao Agent 4, ali sa zvaničnim Stockfish NNUE modelom dostupnim na <https://github.com/official-stockfish/networks>. Služi kao gornja granica poredjenja.

Turnir: Svaki par agenata igra 100 partija (50 kao beli, 50 kao crni igrač) pri jednakom maksimalnom vremenskom budžetu od 1 sekunde po potezu. Beleže se broj pobeda, poraza i nerešenih partija, prosečna dužina partije u potezima, kao i prosečno vreme po potezu.

5 Tehnologije

Python 3 - Glavni programski jezik

python-chess - Reprezentacija table, legalnih poteza i FEN parsiranje

PyTorch - Implementacija NNUE mreže

nnue-pytorch - Treniranje Stockfish NNUE mreže (easytrain.py)

stockfish - Već postojeći šahovski engine koji koristi NNUE mrežu

matplotlib - Vizualizacija metrika i rezultata

numpy - Omogućava brzu manipulaciju podacima i matematičke operacije

6 Gotova rešenja

Stockfish - <https://github.com/official-stockfish/Stockfish>

Lichess - <https://lichess.org/source>

LeelaChessZero - <https://github.com/LeelaChessZero/lc0>

7 Literatura

1. Prezentacije sa predavanja
2. <https://www.quora.com/How-do-I-get-started-with-my-own-chess-engine>
3. <https://www.chess.com/terms/fen-chess>
4. <https://www.chessprogramming.org/NNUE>
5. https://www.chessprogramming.org/Stockfish_NNUE
6. <https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html#quantization-101-and-its-importance>
7. <https://chess.stackexchange.com/questions/33691/halfkp-structure-for-stockfish-shogi-nnue-how-does-it-work>
8. https://www.chessprogramming.org/Principal_Variation_Search
9. https://www.chessprogramming.org/Null_Window
10. https://www.chessprogramming.org/Node_Types#PV
11. https://www.chessprogramming.org/Monte-Carlo_Tree_Search

12. <https://www.chessprogramming.org/Centipawns>
13. https://en.wikipedia.org/wiki/Chess_piece_relative_value
14. <https://www.chessprogramming.org/Evaluation>
15. https://en.wikipedia.org/wiki/Mean_squared_error
16. [https://github.com/official-stockfish/nnue-pytorch/wiki/Basic-training-procedure-\(train.py\)](https://github.com/official-stockfish/nnue-pytorch/wiki/Basic-training-procedure-(train.py))